Observability 100: Using Elastic and OpenTelemetry to Observe Kubernetes

Installing OpenTelemetry in Kubernetes

We have our application stack running on Kubernetes. Now let's observe it using Elastic!

Install the OpenTelemetry Operator

With the advent of the OpenTelemetry Operator and related Helm chart, you can now easily deploy an entire observability signal collection package for Kubernetes, inclusive of: * application traces, metrics, and logs * infrastructure traces (nginx), metrics and logs * application and infrastructure metrics

1. button label="Elastic"
2. Click `Add Data` in bottom-left pane
3. Click `Kubernetes`
4. Click `OpenTelemetry (Full Observability)`
5. Click `Copy to clipboard` below `Add the OpenTelemetry repository to Helm`
6. button label="Terminal"
7. Paste and run helm chart command
8. button label="Elastic"
9. Click `Copy to clipboard` below `Install the OpenTelemetry Operator`
10. button label="Terminal"
11. Paste and run k8s commands to install OTel operator

Checking the Install

First, list out the available namespaces:

```
kubectl get namespaces
```

And get a list of pods running in the `opentelemetry-operator-system` namespace:

```
kubectl -n opentelemetry-operator-system get pods
```

And let's look at the logs from the daemonset collector to see if it is exporting to Elasticsearch without error...

Checking Observability

Let's confirm what signals are coming into Elastic.

First, let's check for logs. Navigate to the button label="Elastic" tab and click on `Observability` > `Discover`.

Next, let's check for infrastructure metrics. Navigate to the button label="Elastic" tab and click on `Infrastructure` > `Hosts`.

Finally, let's check for application traces. Navigate to the button label="Elastic" tab and click on `Applications` > `Service Inventory`. Note that there is not yet any APM data flowing in.

Let's figure out what's up.

Debugging OpenTelemetry in Kubernetes

Debugging APM

Let's describe one of our application pods... Specifically, the monkey pod:

```
kubectl -n trading-1 describe pod monkey
```

Look at the environment variables section; note that there are not yet any OTel environment variables loaded into the pod.

It turns out that after you apply the OTel Operator to your Kubernetes cluster, you need to restart all of your application services:

```
kubectl -n trading-1 rollout restart deployment
```

Now let's wait for our monkey pod to restart:

```
kubectl -n trading-1 get pods
```

And once it has restarted, let's describe it again:

```
kubectl -n trading-1 describe pod monkey
```

And now we can see OTel ENV vars being injected into the monkey pod. Let's check if we have APM data flowing in. Navigate to the button label="Elastic" tab and click on `Applications` > `Service Inventory`. Ok cool, this is starting to look good.

Why is router not showing up?

Click on the `trader` app and look at the `POST /trade/request` transaction. Scroll down to the bottom (trace samples) and look at the waterfall graph. Notice the broken trace. It looks like perhaps one of our applications is not being instrumented. Click on the `POST` span and look at `attributes.service.target.name`. Note that this `POST` is intended to target the `router` service, yet we don't see the `router` service in our Service Map.

Let's look at our `router` pod and see if we can figure out what's up.

```
kubectl -n trading-1 describe pod router
```

Huh. no OTel ENVs, even though the pod was restarted. Let's have a look at that deployment yaml. Click on the button label="Code" button and examine the deployment yaml. Look at the deployment yaml for the `trader` service and compare it to the `router` service. Notice anything missing?

In order for the Operator to attach the correct APM agent, you need to apply an appropriate annotation to each pod. Note that the router pod is missing an annotation. Let's add it. Navigate to the button label="Source" tab and open `router.yaml`.

Uncomment the `instrumentation.opentelemetry.io/inject-nodejs` directive:

```
spec:
  template:
    metadata:
      annotations:
        instrumentation.opentelemetry.io/inject-nodejs: "opentelemetry-operator-system/elastic-instrumentation"
      ...
```

And then reapply the yaml:

```
./build.sh -d force -s router
```

Note that `router` was reconfigured:

```
deployment.apps/router configured
```

Wait for the `router` pod to restart:

```
kubectl -n trading-1 get pods
```

And once it has restarted, let's describe it again:

```
kubectl -n trading-1 describe pod router
```

And now it looks like our OTel ENVs are getting injected as expected. Let's check Elasticsearch.

Navigate to the button label="Elastic" tab and click on `Applications` > `Service Inventory`. Note that we can now see a full distributed trace, as expected!

Automatic Instrumentation

Automatic Instrumentation

Currently, we are injecting the OTel SDK into our applications using the Kubernetes OTel Operator. Let's say you aren't using the Kubernetes OTel Operator, or Kubernetes at all. How do you attach the OTel SDK to your applications?

Java

Attaching the Java OTel SDK at runtime is similar to other languages supported by OTel.

Navigate to the button label="Dockerfile".

We've already made a few changes to note: 1. manually download the EDOT Java SDK:

```
ARG EDOT_VERSION=1.5.0
RUN wget -O edot-javaagent.jar https://repo1.maven.org/maven2/co/elastic/otel/elastic-otel-javaagent/$EDOT_VERSION/elastic-otel-javaagent-$EI
```

2. manually inject the SDK at runtime:

```
ENTRYPOINT ["java", \
"-javaagent:edot-javaagent.jar", \
...
```

Let's verify that we are receiving traces from this service since we switched from using the Kubernetes OTel Operator to manually injecting the OTel SDK:

1. Open the button label="Elasticsearch" tab
2. Click `Discover` in the left-hand navigation pane
3. Set the time picker to show the last 15 minutes
4. Execute the following query:

```
FROM traces-*
| WHERE service.name == "recorder-java"
```

uh-oh; it looks like we stopped receiving traces from the `recorder-java` service when we switched to manual injection. Let's debug and see what's going on.

Debugging

Let's have a look at the logs coming into Elastic:

1. Open the button label="Elasticsearch" tab
2. Click `Discover` in the left-hand navigation pane
3. Set the time picker to show the last 15 minutes
4. Execute the following query:

```
FROM logs-*
| WHERE service.name == "recorder-java"
| WHERE body.text LIKE "* ERROR *"
```

Ah - it looks like the OTel SDK in the `recorder-java` service can't export telemetry to a collector. Why is that?

We need to tell the OTel SDK where to send span data.

1. Navigate to the button label="Dockerfile"
2. Uncomment the line `ENV OTEL_EXPORTER_OTLP_ENDPOINT=http://opentelemetry-kube-stack-daemon-collector.openteleme`
3. Rebuild:

```
./build.sh -d force -b true -s recorder-java -l true
```

Wait for the `recorder-java` pod to restart:

```
kubectl -n trading-1 get pods
```

And once it has restarted, let's describe it again:

```
kubectl -n trading-1 describe pod recorder-java
```

Note the presence of `OTEL_EXPORTER_OTLP_ENDPOINT`.

Let's check if we are receiving span data:

1. Open the button label="Elasticsearch" tab
2. Click `Discover` in the left-hand navigation pane
3. Set the time picker to show the last 15 minutes
4. Execute the following query:

```
FROM traces-*
| WHERE service.name == "recorder-java"
```

Indeed, we are now receiving span data.

Adding Manual Instrumentation

Setting Up Manual Instrumentation

Adding Dependencies

1. Navigate to the button label="Source" tab and open `src/recorder-java/pom.xml`.

Note that we are explicitly linking the OTel SDK to our code at build-time rather than run-time.

Instantiating

1. Navigate to the button label="Source" tab and open `src/recorder-java/src/main/java/com/example/recorder/Main.java`.

Note that we are initializing a global tracer object we can later use to manually create spans.

Adding instrumentation

Annotations

For languages that support annotations (e.g., Python and Java), the OTel SDK lets you instrument with annotations.

Modify `src/recorder-java/src/main/java/com/example/recorder/TradeService.java` like

```
    @WithSpan
    public void auditCustomer(@SpanAttribute(Main.ATTRIBUTE_PREFIX + "customerId") String customerId) {
        log.info("trading for " + customerId);
    }
```

Manual Span Creation

Modify `src/recorder-java/src/main/java/com/example/recorder/TradeService.java` like

```
    public void auditSymbol(String symbol) {
        Span span = tracer.spanBuilder("auditSymbol").startSpan();
        try (Scope ignored = span.makeCurrent()) {
            span.setAttribute(Main.ATTRIBUTE_PREFIX + "symbol", symbol);
             log.info("trading symbol" + symbol);
        } finally {
            span.end();
        }
    }
```

Rebuilding

Rebuild and deploy the `recorder-java` service:

```
./build.sh -d force -b true -s recorder-java -l true
```

And let's recheck our `recorder-java` service.

1. Navigate to the button label="Elastic" tab and click on `Applications` > `Service Inventory`
2. Click on the `recorder-java` service
3. Select the `POST /record` transaction
4. Click on the `auditCustomer` span

5. Note that `auditCustomer` has `customerId` as a span attribute
6. Close the flyout
7. Click on the `auditSymbol` span
8. Note that `auditSymbol` has `symbol` as a span attribute
